Bahir Dar University

Bahir Dar Institute of Technology

Faculty of computing department of software engineering

Course name: operating system and system programming

Title: Ubuntu server and virtualization

Section: A

Submitted by

Name: yordanoskibret

Id: 1702835

Submitted to: Mr Wondimu Baye

Submition date: 20/08/2018

c. Requirements

To successfully install and operate Ubuntu 22.04 Server in a virtual environment, certain software and hardware requirements must be met. The choice of filesystem also plays a crucial role in system performance and data integrity.

i. Which filesystem support (NTFS, FAT32, exFAT, ext4, Btrfs, ZFS, HFS+, APFS) and why

For Ubuntu 22.04 Server, the recommended and most commonly used filesystem is **ext4** (Fourth Extended Filesystem). ext4 is the default filesystem for most Linux distributions due to its robustness, journaling capabilities, and performance. It is a successor to ext3 and offers several improvements, including larger file system and file size limits, more efficient space allocation, and faster file system checking.

While other filesystems like Btrfs and ZFS offer advanced features such as snapshots, data integrity, and volume management, they are often considered more complex for a standard server installation and might require more expertise to manage. NTFS, FAT32, and exFAT are primarily Windows-native filesystems and are generally used for interoperability with Windows systems or for external storage, not as the primary filesystem for a Linux server. HFS+ and APFS are Apple-specific filesystems.

ii. Software

- **Virtualization Software**: Oracle VM VirtualBox, VMware Workstation, or similar virtualization platforms.
- **Ubuntu 22.04 Server ISO Image**: The official installation image for Ubuntu 22.04 LTS (Long Term Support).
- **Host Operating System**: A stable operating system (e.g., Windows, macOS, Linux) capable of running the chosen virtualization software.

iii. Hardware

Minimum recommended hardware for Ubuntu 22.04 Server in a virtualized environment [1]:

- **Processor**: 2 GHz dual-core processor (or equivalent virtual CPUs).

- **RAM**: 2048 MiB (2 GB) RAM for virtualized installs. For physical installs, 4096 MiB (4 GB) is recommended.
- **Disk Space**: 25 GB of hard drive space.
- **Network Adapter**: A virtual network adapter configured for internet access.

d. Installation Steps: Ubuntu 22.04 Server in a Virtual Environment

This section outlines the general steps for installing Ubuntu 22.04 Server within a virtual machine environment, such as Oracle VM VirtualBox or VMware Workstation. While the specifics might vary slightly depending on the virtualization software, the core process remains consistent.

1. **Download Virtualization Software**: If not already installed, download and install your preferred virtualization software (e.g., VirtualBox from virtualbox.org).
2. **Download Ubuntu 22.04 Server ISO**: Obtain the official Ubuntu 22.04 LTS Server ISO image from the Ubuntu website.
3. **Create a New Virtual Machine**: Open your virtualization software and create a new virtual machine. Configure the following settings:
    - **Name**: Give your VM a descriptive name (e.g., “Ubuntu2204Server”).
    - **Machine Folder**: Choose a location on your host machine to store the VM files.
    - **Type**: Linux
    - **Version**: Ubuntu (64-bit)
    - **Memory (RAM)**: Allocate at least 2048 MB (2 GB) of RAM. More is better if your host system allows.
    - **Processors**: Allocate at least 2 virtual CPUs.
    - **Hard Disk**: Create a virtual hard disk with at least 25 GB of space. Choose VDI (VirtualBox Disk Image) for VirtualBox or the equivalent for other software, and select “Dynamically allocated” for efficient space usage.

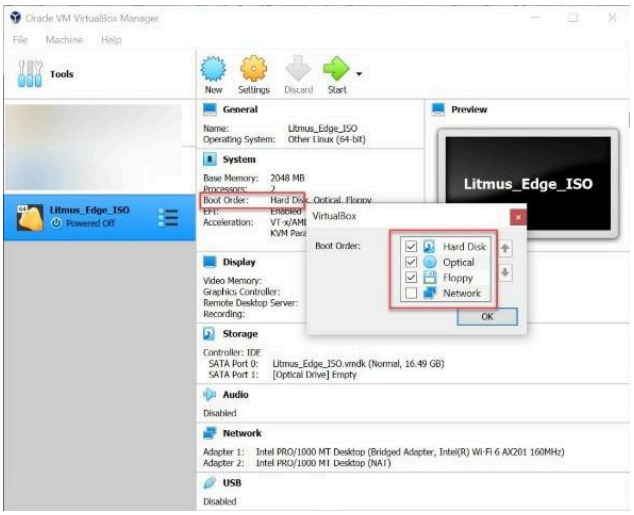

Figure 1: VirtualBox Manager with VM settings for Ubuntu 22.04 Server.

4. **Mount the ISO Image**: In the VM settings, navigate to the “Storage” section. Under the optical drive, select the downloaded Ubuntu 22.04 Server ISO image as the virtual CD/DVD drive.

Figure 2: Dialog for selecting the start-up disk in VirtualBox.

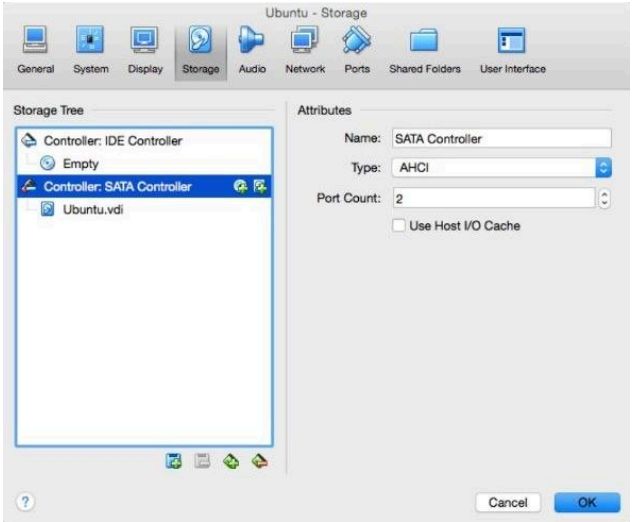

Figure 3: VirtualBox storage settings, showing the attached Ubuntu VDI and SATA controller.

5. **Start the Virtual Machine**: Start the newly created virtual machine. It should boot from the mounted ISO image.

6. **Language Selection**: Choose your preferred language for the installation.

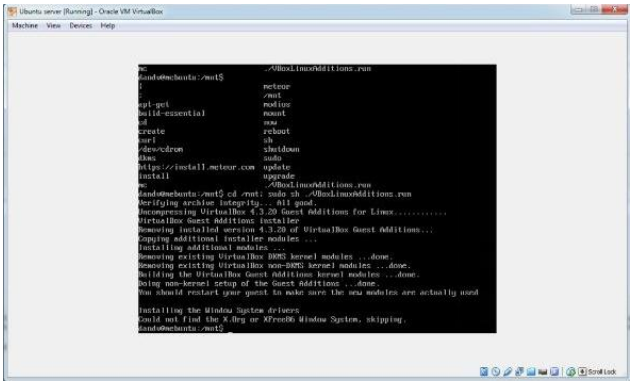

Figure 4: Ubuntu 22.04 Server installer language selection.

7. **Keyboard Configuration**: Select your keyboard layout.

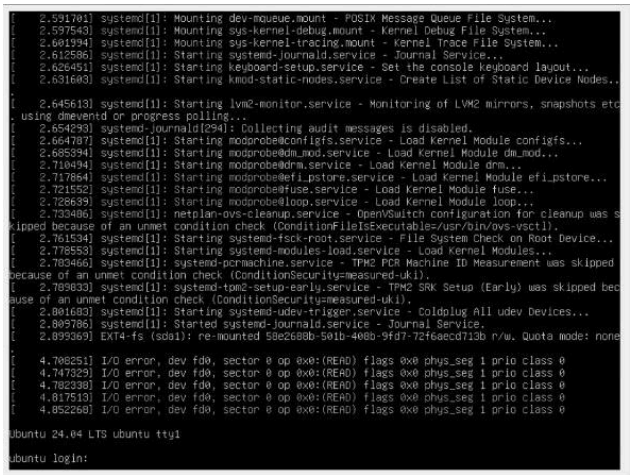

Figure 5: Keyboard layout configuration during Ubuntu installation.

8. **Network Configuration**: Configure the network. DHCP is usually sufficient for most virtual environments. Ensure the VM has internet access.

Figure 6: Network configuration screen.

9. **Storage Configuration**: This is a critical step. For a basic installation, you can choose to “Use an entire disk” and let the installer automatically partition the virtual hard disk. For more advanced setups, you can manually partition the disk, creating partitions for `/`, `/boot`, and `swap`.

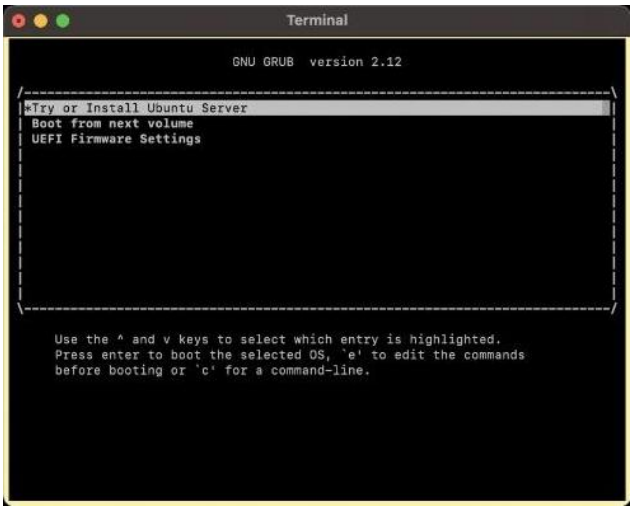

Figure 9: Guided storage setup for disk partitioning.

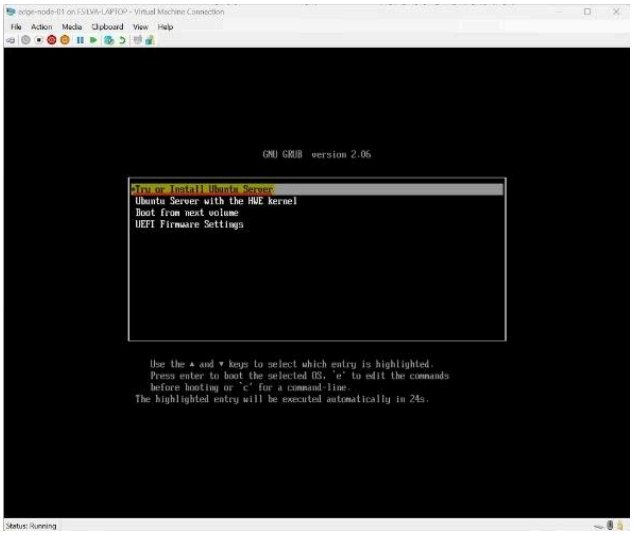

Figure 10: Summary of the proposed storage configuration.

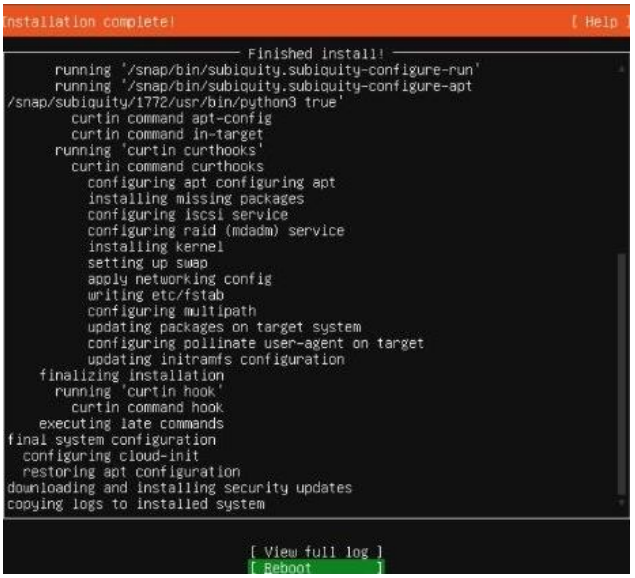

Figure 11: Confirmation prompt for destructive disk partitioning action.

10. **Profile Setup**: Create a user account with a strong password. This will be your primary user for logging into the server.

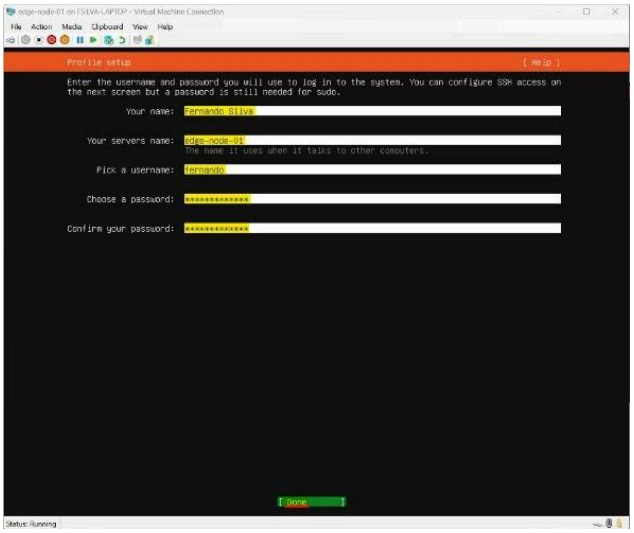

Figure 7: User profile setup screen.

11. **SSH Setup**: It is highly recommended to install the OpenSSH server during installation. This allows you to remotely connect to your server using SSH, which is essential for server management.

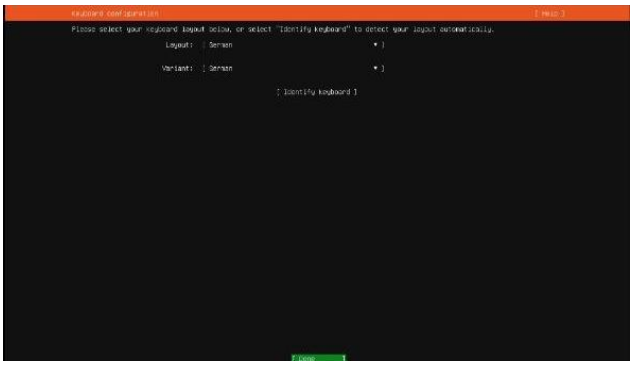

Figure 8: OpenSSH server installation option.

12. **Featured Server Snaps**: Select any additional server snaps you might need (e.g., `docker`, `microk8s`). For a minimal installation, you can skip this step.

Figure 9: Selection of featured server snaps.

13. **Installation Process**: The installer will now proceed with copying files and installing the system. This may take some time.

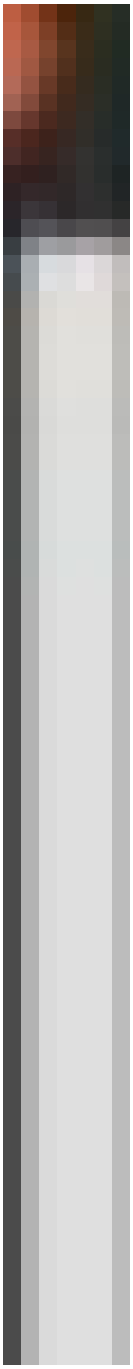

Figure 10: Installation progress screen.

14. **Reboot**: Once the installation is complete, you will be prompted to reboot. Remember to unmount the ISO image from the virtual optical drive before rebooting to prevent booting back into the installer.

Figure 11: Installation complete, prompting for reboot.

15. **Login**: After rebooting, you should see the login prompt. Enter the username and password you created during the profile setup.

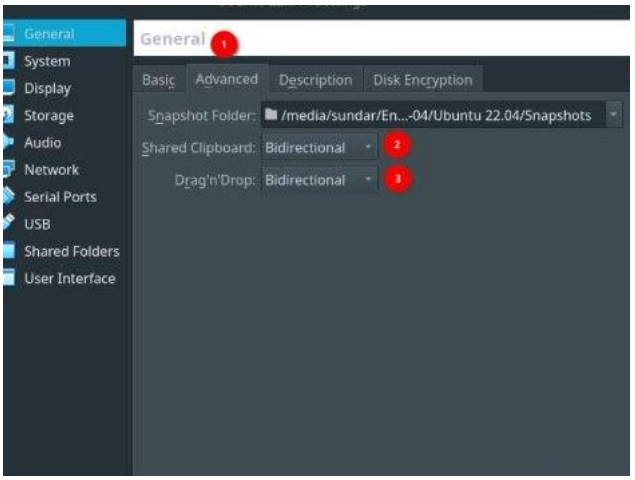

Figure 12: Login prompt after successful installation.

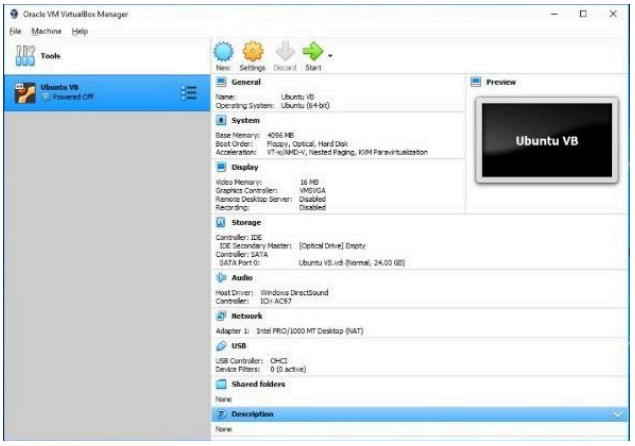

Figure 13: Successful login to the Ubuntu server.

e. Issues (Problems Faced)

During the installation and initial configuration of Ubuntu 22.04 Server in a virtual environment, several common issues may arise:

- **Virtual Machine Not Booting from ISO**: The VM might not recognize the ISO image, leading to a boot failure or booting into an empty shell.

- **Network Connectivity Issues**: The virtual machine might not obtain an IP address or connect to the internet, hindering package installations and updates.
- **Disk Partitioning Errors**: Incorrect partitioning during installation can lead to insufficient space for the OS or failure to boot.
- **Slow Performance**: The VM might experience sluggish performance due to insufficient allocated resources (RAM, CPU) or incorrect virtualization settings.
- **SSH Connection Refused**: After installation, attempts to connect via SSH might fail, indicating issues with the OpenSSH server installation or firewall rules.
- **Outdated Software/No LTS/No Hardware Support**: The prompt mentions issues with an outdated OS having no long-term support (LTS) and no hardware support. Ubuntu 22.04 LTS addresses these concerns by providing five years of maintenance updates until April 2027, ensuring long-term stability and security. It also includes broad hardware support, which is continuously updated with new kernel versions.

f. Solution:

- **Virtual Machine Not Booting from ISO**: Ensure the ISO image is correctly mounted in the virtual machine's storage settings and that the boot order prioritizes the optical drive or the virtual hard disk where the ISO is attached. Verify the ISO image integrity.
- **Network Connectivity Issues**: Check the virtual network adapter settings (e.g., set to NAT or Bridged Adapter). Ensure the host machine has internet access. Within the VM, verify network configuration files (e.g., `/etc/netplan/*.yaml`) and restart the network service (`sudo netplan apply`).
- **Disk Partitioning Errors**: During installation, carefully review the partitioning scheme. For beginners, choosing "Use an entire disk" is often the safest option. For manual partitioning, ensure `/` (root) and `/boot` partitions are correctly sized and formatted.
- **Slow Performance**: Increase the allocated RAM and CPU cores for the virtual machine in the virtualization software settings. Ensure that hardware virtualization features (e.g., Intel VT-x, AMD-V) are enabled in the host machine's BIOS/UEFI settings.

- **SSH Connection Refused**: During installation, ensure OpenSSH server is selected. If not, install it post-installation (`sudo apt update && sudo apt install openssh-server`). Check firewall status (`sudo ufw status`) and allow SSH traffic (`sudo ufw allow ssh`). Verify the SSH service is running (`sudo systemctl status ssh`).
- **Outdated OS/No LTS/No Hardware Support**: By choosing Ubuntu 22.04 LTS, these issues are mitigated. Regularly update the system (`sudo apt update && sudo apt upgrade`) to receive the latest security patches and software improvements. For specific hardware issues, consult Ubuntu's official documentation or community forums.

g. Filesystem Support

As discussed in the requirements section, ext4 is the recommended filesystem for Ubuntu 22.04 Server due to its reliability, performance, and journaling capabilities. It is well-suited for server environments, providing a balance of features and stability. Other filesystems like XFS are also viable options for specific use cases, particularly when dealing with very large files or high-performance storage, but ext4 remains the default and most common choice for general-purpose server installations.

h. Advantage and Disadvantage

This section will discuss the general advantages and disadvantages of using Ubuntu 22.04 Server in a virtualized environment.

Advantages:

- **Cost-Effective**: Reduces hardware costs by running multiple virtual servers on a single physical machine.
- **Resource Optimization**: Maximizes the utilization of underlying hardware resources.
- **Isolation**: Provides a secure and isolated environment for each server instance, preventing conflicts between applications.

- **Flexibility and Portability**: Virtual machines can be easily moved, copied, and deployed across different physical hosts.
- **Snapshots and Rollbacks**: Ability to create snapshots of the VM state, allowing for easy rollbacks in case of issues.
- **Disaster Recovery**: Simplifies backup and recovery processes, enhancing business continuity.
- **Learning Environment**: Ideal for learning and experimenting with server configurations without affecting a production system.

Disadvantages:

- **Performance Overhead**: Virtualization introduces a slight performance overhead compared to running directly on bare metal.
- **Resource Contention**: If not properly managed, multiple VMs can contend for resources, leading to performance degradation.
- **Single Point of Failure**: The physical host becomes a single point of failure; if it goes down, all hosted VMs are affected.
- **Management Complexity**: Managing a virtualized environment can be more complex than managing a single physical server, requiring specialized knowledge of virtualization software.
- **Licensing Costs**: While open-source virtualization solutions exist, some enterprise-grade hypervisors and tools can incur licensing costs.

i. Conclusion

This documentation has explored the essential aspects of setting up and understanding Ubuntu 22.04 Server in a virtualized environment, delving into the intricacies of the `chroot()` system call, and examining the broader context of virtualization and UNIX standardization. We have outlined the practical steps for installation, discussed common challenges and their solutions, and highlighted the benefits and drawbacks of virtualization.

The choice of Ubuntu 22.04 LTS provides a stable and well-supported platform for server operations, mitigating concerns about outdated software and lack of long-term support. Virtualization, with its ability to optimize resources, enhance flexibility, and

improve disaster recovery, stands as a cornerstone of modern IT infrastructure. The `chroot()` system call, while not a complete security solution, offers valuable isolation for specific tasks and contributes to system maintenance and development workflows.

Finally, UNIX standardization, through POSIX and SUS, ensures a consistent and portable operating environment, fostering innovation and compatibility across diverse systems.

j. Future Outlook/Recommendation

For future endeavors, it is recommended to explore containerization technologies like Docker and Kubernetes, which build upon the principles of isolation and resource efficiency offered by virtualization but provide even lighter-weight and more agile deployment options. Further investigation into advanced filesystem features, such as those offered by Btrfs and ZFS, could also be beneficial for specific data management and integrity requirements.

Additionally, continuous learning and staying updated with the latest security practices for both the operating system and the virtualization platform are crucial for maintaining a robust and secure server environment. Experimenting with automation tools for deployment and configuration management (e.g., Ansible, Puppet, Chef) can further streamline server administration tasks and improve efficiency.

2. Briefly explain the what, why, and how virtualization in modern operating system.

What is Virtualization?

Virtualization is a technology that allows for the creation of virtual (rather than physical) versions of computing resources, such as servers, storage devices, networks, and even operating systems. It abstracts the underlying physical hardware, enabling

multiple virtual instances to run concurrently on a single physical machine [2]. Each virtual instance operates as an independent entity, with its own operating system and applications, isolated from other virtual instances and the host hardware.

Why is Virtualization Used?

Virtualization has become a cornerstone of modern IT infrastructure due to several compelling advantages [3]:

- **Resource Utilization**: Physical servers are often underutilized. Virtualization allows multiple virtual machines (VMs) to share the resources of a single physical server, significantly increasing hardware utilization and reducing the need for additional physical hardware.
- **Cost Reduction**: By maximizing hardware utilization, organizations can reduce capital expenditures on physical servers, as well as operational costs related to power consumption, cooling, and data center space.
- **Isolation and Security**: VMs are isolated from each other, meaning an issue in one VM typically does not affect others. This enhances security and stability, as applications can be compartmentalized.
- **Disaster Recovery and Business Continuity**: VMs can be easily backed up, replicated, and restored, facilitating faster disaster recovery and improving business continuity. In case of a hardware failure, VMs can be migrated to healthy physical servers with minimal downtime.
- **Flexibility and Scalability**: Virtual environments offer greater flexibility to provision and de-provision resources as needed. VMs can be quickly deployed, scaled up or down, and migrated across physical hosts, adapting to changing workload demands.
- **Development and Testing**: Virtualization provides isolated environments for development and testing, allowing developers to create and test applications without impacting production systems or requiring dedicated hardware.

How Virtualization Works?

At the heart of virtualization is a software layer called a **hypervisor** (also known as a Virtual Machine Monitor or VMM). The hypervisor sits directly on the physical hardware

(Type 1 or bare-metal hypervisor) or runs as an application on top of a host operating system (Type 2 or hosted hypervisor) [4].

The hypervisor performs several key functions:

- **Hardware Abstraction**: It abstracts the physical hardware resources (CPU, memory, storage, network) and presents them as virtual resources to each VM.
- **Resource Management**: It allocates and manages the physical resources among the various VMs, ensuring that each VM receives the necessary resources to operate.
- **Isolation**: It ensures that each VM operates independently and securely, preventing interference between them.
- **Emulation/Para-virtualization**: Depending on the type, the hypervisor either fully emulates the hardware for the guest OS (making it unaware it's running virtually) or uses para-virtualization, where the guest OS is modified to communicate directly with the hypervisor for better performance.

Each VM consists of a guest operating system and its applications, all encapsulated within a set of files on the host system. When a VM is started, the hypervisor allocates a portion of the physical resources to it, and the guest OS boots up as if it were running on dedicated hardware.

Comparison of Hypervisor Types

Feature	Type 1 (Bare-Metal)	Type 2 (Hosted)
Installation	Directly on physical hardware	On top of a host OS
Performance	High (near bare-metal)	Moderate (due to host OS overhead)
Security	High (smaller attack surface)	Moderate (dependent on host OS security)
Use Cases	Enterprise data centers, Cloud computing	Development, Testing, Personal use
Examples	VMware ESXi, Microsoft Hyper-V, Xen	Oracle VM VirtualBox, VMware Workstation

3. UNIX Standardization and Implementation. Briefly describe and explain about the `chroot()` system call.

UNIX History and Standardization

The history of UNIX dates back to the mid-1960s at Bell Labs, where Ken Thompson, Dennis Ritchie, and others developed the first version of UNIX after the Multics project was abandoned. Over the decades, UNIX evolved through several major versions and branches, leading to the need for standardization to ensure interoperability and portability [5].

Major UNIX Branches and Implementations

- **Research UNIX**: The original versions developed at Bell Labs (e.g., Version 7 UNIX).
- **BSD (Berkeley Software Distribution)**: Developed at UC Berkeley, introducing key features like TCP/IP networking. Modern descendants include FreeBSD, OpenBSD, and NetBSD.
- **System V**: Developed by AT&T, it became the basis of many commercial UNIX systems like AIX, Solaris, and HP-UX.
- **Linux**: A UNIX-like kernel developed by Linus Torvalds in 1991. While not technically UNIX (as it doesn't share the original code), it adheres to UNIX principles and standards.

Standardization Efforts

- **POSIX (Portable Operating System Interface)**: Developed by the IEEE, POSIX defines a standard operating system interface and environment, including a command interpreter (shell) and common utility programs.
- **Single UNIX Specification (SUS)**: Developed and maintained by The Open Group, the SUS builds upon POSIX and is required for a system to be officially

branded as “UNIX”.

The `chroot()` System Call

The `chroot()` system call, short for “change root,” is a fundamental operation in UNIX and UNIX-like operating systems that changes the apparent root directory for the current running process and its children [6]. When a process is `chrooted`, it cannot access files and directories outside of the new designated root directory, effectively creating an isolated environment often referred to as a “chroot jail” [7].

Practical Example in C

The following C code demonstrates how to use the `chroot()` system call to change the root directory of a process. Note that this operation requires root privileges.

```c
#include <stdio.h>
#include <unistd.h>
#include <dirent.h>

int main(void) {
    // 1. Define the new root path
    const char *new_root = "/tmp/chroot_jail";

    // 2. Change the current working directory to the new root
    if (chdir(new_root) != 0) {
        perror("chdir failed");
        return 1;
    }

    // 3. Call chroot() to change the root directory
    if (chroot(new_root) != 0) {
        perror("chroot failed (must be run as root)");
        return 1;
    }

    // 4. Verify the change by listing the content of the new root (/)
    printf("Successfully changed root to: %s\n", new_root);
    printf("Content of the new root (/):\n");

    struct dirent *de;
    DIR *dr = opendir("/");
    if (dr == NULL) {
        perror("opendir failed");
        return 1;
    }

    while ((de = readdir(dr)) != NULL) {
        printf(" %s\n", de->d_name);
    }
    closedir(dr);

    // 5. Attempt to access a file outside the jail (will fail)
    if (access("/etc/passwd", R_OK) != 0) {
        printf("\nAccess to /etc/passwd denied (as expected).\n");
    } else {
        printf("\nWarning: Access to /etc/passwd still possible!\n");
    }
```

```
    return 0;
}
```

How `chroot()` Works

When `chroot("new_root_path")` is called, the operating system changes the root directory (`/`) for the calling process and any processes it subsequently spawns to `new_root_path`. From the perspective of the `chrooted` process, `new_root_path` becomes `/`. Any attempt to access files outside this new root will fail, as the process's view of the filesystem is confined.

Advantages and Disadvantages

Advantages:

- **Isolation**: Provides a basic level of filesystem isolation for processes.
- **Security Enhancement**: Can limit the scope of damage from a compromised application.
- **System Maintenance**: Facilitates system recovery, repair, and package building.
- **Simplicity**: Relatively simple to implement and use compared to full virtualization or containerization.

Disadvantages:

- **Not a Security Panacea**: A `chroot` jail is not impenetrable, especially if the `chrooted` process retains root privileges.
- **Complexity in Setup**: Setting up a functional `chroot` environment can be complex, as all necessary libraries, executables, and configuration files must be present within the new root.
- **Limited Scope**: Only isolates the filesystem; it does not isolate other system resources like memory, network interfaces, or process IDs in the same way that containers or virtual machines do.
- **Management Overhead**: Maintaining and updating `chroot` environments can add administrative overhead.

References

[1] Ubuntu Server System Requirements. https://ubuntu.com/server/docs/reference/installation/system-requirements/ [2] What Is Virtualization? IBM. https://www.ibm.com/think/topics/virtualization [3] What is virtualization? Red Hat. https://www.redhat.com/en/topics/virtualization/what-is-virtualization [4] What is Virtualization? The Complete Guide. Nutanix. https://www.nutanix.com/info/virtualization [5] History of Unix. Wikipedia. https://en.wikipedia.org/wiki/History_of_Unix [6] chroot(2) - Linux manual page. https://man7.org/linux/man-pages/man2/chroot.2.html [7] Chroot Simplified: A Beginner's Guide to Secured Linux. Lenovo. https://www.lenovo.com/us/en/glossary/what-is-chroot/ [8] Is chroot a security feature? Red Hat. https://www.redhat.com/en/blog/chroot-security-feature [9] chroot — A Linux Wonder! Medium. https://medium.com/100-days-of-linux/chroot-a-linux-wonder-fc36ed08087e